

Trace — Habit & Task Tracker

Project Spec (Source of Truth)

1. The Idea, In One Sentence

An offline-first habit and task tracker that helps you understand not only what you missed, but why you missed it.

Not another CRUD habit tracker. Not "Flutter app with streaks and a heatmap." The product's entire reason to exist is one gap no mainstream app fills.

2. Why This Gap Is Real (Research Summary)

Researched across Habitica, Loop Habit Tracker, TickTick, Todoist, Habitify, Atoms (James Clear's app), Microsoft To Do, Google Tasks, Notion, and Obsidian — reviews, Play Store/App Store feedback, Reddit sentiment, and forum discussions.

The core gap:

- Apps either do habits well with weak/no task management (Loop), or bolt a shallow habit module onto a task app (TickTick caps free habits at ~5).
- Nobody merges both at real depth in one app.
- Every tracker shows *that* a streak broke. None show *why* — except Obsidian, and only by accident, because a user's habit log happens to sit next to their daily notes. That requires a full personal-knowledge-management setup most people will never build. No dedicated, native app does this.

Other repeated, validated pain points:

- Reminders and calendar view frequently locked behind premium (Todoist, TickTick) — the single most-repeated free-tier complaint found across every app reviewed.
- Subscription fatigue — users explicitly prefer one-time payment, ads, or fully free over recurring fees for something this simple.
- Streak-based, all-or-nothing tracking causes real anxiety. A whole micro-category of apps now exists specifically to counter this (weekly targets instead of daily perfection, no punitive notifications, cumulative counts over consecutive streaks).
- Widget reliability is a recurring, named bug class (e.g., HabitNow's widget desyncing from app state) — not a one-off complaint.
- Nothing bridges offline-first with optional sync. You get Loop (offline, no sync, Android-only) or subscription cloud apps (TickTick, Todoist). Nothing sits between those.

- Notion and Obsidian, when repurposed for this, fail the same way every time: the *system* becomes the work. Steep learning curve, no native recurring tasks, formula overhead for things as simple as a progress bar — "productivity theater" instead of actual productivity.

Design conclusion drawn from the research: the failure mode to avoid isn't "too few features" — it's "using the app becomes its own task." Minimalism is a product principle here, not just a style choice.

3. Product Philosophy

- Collect data early, analyze data later. The "why" prompt is cheap to build and should ship in v0.1. Pattern detection needs weeks of real data and should NOT be built until that data exists.
 - Progressive disclosure. The home screen shows only today's habits and today's tasks. Nothing else. Heatmaps, stats, and analytics live one tap away, never on the main screen.
 - Gentle by default. Weekly targets are a first-class option, not a workaround. No guilt-tripping notifications. A missed day doesn't erase a long streak's cumulative value.
 - No feature is free-tier-gated. Reminders, calendar view, and habit/task count are never paywalled — that's the exact complaint this app exists to not repeat.
 - Offline-first, always. The app works completely without internet. Sync (when it eventually exists) is an enhancement layered on top of the local database, never a dependency.
-

4. Features By Version

v0.1 — MVP (Resume-worthy, Play Store-ready)

Dashboard

- Clean welcome screen
- Today's habits list
- Today's tasks list
- Nothing else on this screen

Habit Module

- Unlimited habits (no cap, ever)
- Full CRUD
- Habit categories
- Flexible scheduling: daily / specific weekdays / every X days / X times per week

- Habit completion tracking
- Streak tracking (alongside cumulative stats, not instead of)

To-Do Module

- Unlimited tasks (no cap, ever)
- Full CRUD
- Due dates
- Priorities
- Categories

Heatmap

- GitHub-style completion heatmap

Statistics

- Completion %
- Total completions
- Current streak
- Longest streak
- Weekly progress
- Monthly progress
- Emphasis on cumulative progress, not just an unbroken streak count

"Why Did You Miss?" — the core differentiator

- Triggered when a habit is marked missed
- Optional, skippable, one tap or one line — never forced
- Preset quick options: Busy / Sick / Traveling / Forgot / Low Motivation / Other
- "Other" opens a free-text field
- **Important: this data is only collected in v0.1, not analyzed. No pattern detection yet.**

Notifications

- Local notifications only
- Not a premium feature — free from day one

Export

- CSV
- JSON

Theme

- Dark mode

Architecture

- Fully offline-first, no login required
-

v0.2 — Usability Pass

- Better/more flexible scheduling
- Weekly goal mode
- Search
- Filters
- Calendar view improvements

No AI. No cloud. Still purely local.

v0.3 — Now Analyze the "Why" Data

By this point there's real historical data to work with. This is the version where the differentiator actually pays off.

- Aggregate reason data (e.g., "Late Night — 7, Travel — 4, Busy — 5")
- Simple pattern surfacing (e.g., "You usually miss workouts after sleeping late," "Mondays have the highest miss rate")
- Trend graphs
- Reason analytics

Still no AI — this is basic aggregation/analysis of data you already collected, not machine learning.

v0.4 — Widgets (Isolated Milestone)

Deliberately its own version, separate from other feature work, because it requires native Android platform-channel code even though the app is built in Flutter.

- Home screen widget
 - Quick habit completion from the widget
 - Widget-to-app data sync (get this right — it's a named failure point in competitor apps)
-

v1.0 — Cloud Ecosystem

- Login
 - Cloud sync
 - Multi-device sync
 - AI-assisted insights / smart suggestions (only once there's a real behavioral dataset to work from)
-

5. Explicitly NOT Building (v0.1-v0.4)

- RPG-style gamification, guilds, social/friends features
 - Health app integrations, WearOS
 - Backend, Firebase, any cloud dependency
 - Any paywalled core feature or subscription requirement
 - AI (until v1.0, and only once justified by real data)
-

6. Tech Stack

Layer	Choice	Why
Framework	Flutter (Dart)	Cross-platform now, keeps iOS optional later without a rewrite; chosen over native Kotlin because the goal is cross-platform capability and general SDE positioning, not Android-specialist roles
State management	Riverpod (with <code>@riverpod</code> code generation)	Confirmed 2026 default for new Flutter projects — compile-time safe, noBuildContext dependency, strong async/testing story
Routing	go_router	Officially backed, integrates cleanly with Riverpod
Local database	Drift (NOT Isar)	Isar was abandoned by its maintainer — long gaps between commits, official v4 never reached stable. Drift is SQL-backed, actively maintained, type-safe, reactive, and the current safe default for a new 2026 app
Charts	fl_chart	Standard choice for heatmap/streak/trend visualizations

Layer	Choice	Why
Calendar	<code>table_calendar</code>	Standard calendar widget
Notifications	<code>flutter_local_notifications</code>	Local-only notifications, no backend needed
Fonts	<code>google_fonts</code>	Simple, no manual font asset management
Widgets (v0.4 only)	<code>home_widget</code> plugin + native Kotlin glue	Flutter can't render directly to the Android home screen; this bridges Dart data to a native widget provider

Note on Riverpod syntax: use the modern `@riverpod` annotation-based code generation, not the older manual provider declarations. This adds a `build_runner watch` step to your dev workflow but is the current standard — get this right from your first screen rather than retrofitting later.

7. Architecture Rules

- Offline-first for everything. No login, no backend, no Firebase, no cloud in v0.1–v0.4.
- All data stored locally via Drift.
- Architecture should allow future cloud sync to be *added on top of* the local database later, not require replacing it.

Folder structure:

```
lib/  
├─ core/  
├─ models/  
├─ database/  
├─ providers/  
├─ routes/  
├─ screens/  
├─ widgets/  
├─ services/  
└─ main.dart
```

8. UX Principle (Non-Negotiable)

Avoid becoming another Notion. Avoid productivity theater.

Home screen should look like this and nothing more:

Good Morning

Today's Habits

☐ Workout

☐ Read

☐ Water

Today's Tasks

☐ Assignment

☐ Buy groceries

No graphs, no achievements, no heatmap, no streak counters on this screen. All of that lives one tap away. This is progressive disclosure, and it directly answers the Notion/Obsidian failure mode identified in research: the system becoming the work.

9. Play Store Practicalities

- One-time registration fee: **\$25**
 - Requires a **non-prepaid card** (PayPal does not work)
 - New personal developer accounts require **closed testing**: roughly **12 testers for ~14 consecutive days** before the app can go public
 - This is a hard blocker on launch timing — line up testers (classmates, friends) early, don't wait until the app is "done"
-

10. Naming

Working name: Trace

- Reads naturally in speech ("I built an app called Trace")
 - Double meaning: habit *tracking*, and *tracing back* to why something happened
 - Common enough as a word that Play Store / domain availability should be checked before committing — fallback: "Trace Habits"
-

11. Resume Positioning

Don't say: "I built a Flutter habit tracker."

Say (or close to it):

"I researched competing habit and task-tracking apps, identified that they record missed habits but never capture the reasons behind them, and designed an offline-first

system that collects lightweight context at the moment a habit is missed — enabling later pattern analysis without adding daily friction for the user."

This framing works for both product-based and service-based companies because it demonstrates competitor research, a specific identified user pain point, and a technical implementation grounded in real data — not just "I followed a tutorial and shipped an app."

A live Play Store app with a handful of real testers is worth more in this pitch than a polished GitHub repo nobody has used.

12. Immediate Next Steps

1. Initialize Git & GitHub repository
2. Set up Riverpod (with code generation)
3. Set up Drift
4. Confirm/explain project folder structure
5. Build the Welcome/Dashboard screen
6. Implement v0.1 feature by feature, in order, following the list in Section 4

This roadmap is the source of truth. Don't re-litigate scope or tech stack again unless real implementation constraints or actual user feedback force a change. Next real output should be code, not another planning pass.